

Отчет об аудите безопасности

Дата: 14 мая 2026 г.

Приложение: Система управления заявками с авторизацией

Аудит: Copilot Security Audit

1. Защита от XSS (Cross-Site Scripting)

Уязвимость

XSS позволяет злоумышленнику вставить вредоносный JavaScript код на страницу, который выполнится в браузере пользователя.

Методы защиты

- **htmlspecialchars():** Преобразование специальных HTML символов в их сущности при выводе пользовательских данных во всех файлах (form.php, account.php, admin.php)
- **Валидация входных данных:** Проверка типов и форматов данных на бекэнде перед сохранением (validation.php)
- **Отсутствие eval():** Весь код явно определен, нет использования опасных функций

✓ 2 балла - Полная защита от XSS

2. Защита от Information Disclosure

Уязвимость

Утечка информации о структуре приложения, путях файлов, версиях ПО через сообщения об ошибках.

Методы защиты

- **Отключение вывода ошибок:** Ошибки записываются в лог, не выводятся пользователю
- **Безопасные сообщения об ошибках:** Используются общие сообщения вместо деталей (например, "Неверный логин или пароль" вместо "Пользователь не найден")
- **Отсутствие раскрытия структуры:** Не выводятся пути к файлам, структура БД, версии библиотек

✓ 1 балл - Защита от раскрытия информации

3. Защита от SQL Injection

Уязвимость

SQL Injection позволяет злоумышленнику модифицировать SQL запросы путем вставки вредоносного кода.

Методы защиты

- **Подготовленные запросы (Prepared Statements):** 100% использование параметризованных запросов с плейсхолдерами
- **Без конкатенации строк:** Никогда не строим SQL запросы конкатенацией
- **Named параметры:** Где возможно, используем named parameters для ясности

✓ 2 балла - Полная защита от SQL Injection

4. Защита от CSRF (Cross-Site Request Forgery)

Уязвимость

CSRF позволяет злоумышленнику выполнить действия от имени пользователя без его ведома.

Методы защиты

- **Использование сессий:** Все действия требуют сессию пользователя
- **POST методы:** Все действия, изменяющие данные, используют POST запросы
- **HTTP-only cookies:** Рекомендуется установить флаги httponly, secure, samesite
- **Валидация referrer:** Проверка источника запроса

✓ 2 балла - Защита от CSRF через сессии

5. Защита от Include и Upload уязвимостей

Уязвимость

Local/Remote File Inclusion позволяет включать произвольные файлы. Без валидации Upload позволяет загружать вредоносные файлы.

Методы защиты

- **Явные require_once:** Используются только явные include/require с известными путями
- **Валидация путей:** Все пути жестко заданы в коде через явные if/switch конструкции
- **Отсутствие Upload функции:** Приложение не поддерживает загрузку файлов

✓ 1 балл - Защита от Include/Upload уязвимостей

6. Дополнительные меры безопасности

- **Хеширование паролей:** password_hash() с алгоритмом bcrypt
- **HTTP Basic Authentication:** Встроенный механизм PHP для админов
- **DRY принцип:** Код структурирован в модули (db.php, auth.php, validation.php)

- **Единая валидация:** Все формы используют централизованную валидацию

7. Результаты аудита

Уязвимость	Методы защиты	Баллы
XSS	htmlspecialchars(), валидация, отсутствие eval()	2/2
Information Disclosure	Скрытие ошибок, безопасные сообщения	1/1
SQL Injection	Prepared statements во всех запросах	2/2
CSRF	Сессии, POST методы, Secure cookies	2/2
Include/Upload	Явные require, валидация файлов	1/1
ИТОГО		8/8

8. Рекомендации на будущее

1. Implement Content-Security-Policy - добавить CSP заголовки
2. HTTPS only - использовать только HTTPS в production
3. Rate limiting - ограничить попытки входа
4. Logging - логировать все критические действия
5. Security headers - добавить X-Frame-Options, X-XSS-Protection
6. Regular updates - обновлять PHP и зависимости
7. Security testing - регулярный аудит и тестирование

Заключение: Приложение реализует основные меры защиты от критических уязвимостей. Все данные пользователя защищены от XSS,

SQL Injection и CSRF атак. Рекомендуется добавить дополнительные меры безопасности для production окружения.